

Nonlocal Transition Kernel for Efficient Learning of Restricted Boltzmann Machines

Kaiji Sekimoto

Graduate School of Science and Engineering, Yamagata University, Japan
sekimoto@yz.yamagata-u.ac.jp

Muneki Yasuda

Graduate School of Science and Engineering, Yamagata University, Japan
muneki@yz.yamagata-u.ac.jp

Abstract

Learning restricted Boltzmann machines (RBMs) is computationally challenging because it requires expectations whose exact evaluation is generally intractable. The expectations are typically evaluated using a sampling approximation based on blocked Gibbs sampling (BGS), which is a local Markov chain Monte Carlo transition kernel. However, the locality of BGS can lead to poor sampling quality when the RBM has high energy barriers, thereby degrading learning performance. Deep tempering (DT), which performs parallel tempering over a sequence of learnable RBMs including the training RBM, alleviates this locality issue. However, DT algorithmically requires multiple steps to move through the RBM sequence to achieve a nonlocal transition. In this paper, we propose a transition kernel defined over the RBM sequence used in DT. The proposed kernel has a round-trip structure over the sequence, enabling nonlocal moves within a single transition while leaving the RBM sequence invariant. Numerical experiments show that the proposed kernel performs nonlocal transitions more frequently and achieves higher sampling quality with fewer transitions than BGS and DT. We further verify that learning based on the proposed kernel is more stable and mitigates the training failures observed with BGS- and DT-based learning.

1 Introduction

A restricted Boltzmann machine (RBM) is a latent-variable energy-based model with a two-layer architecture and has been applied to various tasks such as pretraining deep learning models [1], collaborative filtering [2, 3], hierarchical clustering [4], inferring effective many-body interactions [5], anomaly detection [6, 7, 8], analysis in many-body physics [9, 10], and design of polymer materials [11]. RBM training is performed using maximum likelihood estimation and requires the evaluation of expectations over the RBM. However, the evaluation is generally difficult and requires an approximation method. Accurate approximation is essential for achieving high learning performance. Since blocked Gibbs sampling (BGS), a type of Markov chain Monte Carlo (MCMC), can be easily performed for an RBM, the expectations are typically evaluated using a sampling approximation based on BGS.

The sampling approximation directly depends on the sampling quality, that is, how closely the sample distribution follows the target distribution. Although MCMC theoretically requires infinitely many transition steps to sample from the target distribution, this is impractical, and the transition must be terminated after a finite number of steps. Therefore, in practice, the resulting sample distribution does not exactly match the target distribution, and it is thus desirable for the finite-step sample distribution to approximate the target distribution as closely as possible. This can be achieved through two design choices: (i) an initial

distribution that roughly approximates the target distribution, and (ii) a rapidly mixing transition kernel. The contrastive divergence (CD) method [12] and the persistent CD (PCD) method [13], which are well-known RBM learning methods, corresponds to the former approach. They set the initial distribution to the data distribution and the sample distribution used in the previous parameter update, respectively. The sampling quality of these learning methods also directly depends on the mixing time of BGS. BGS is a local transition kernel that explores states in the neighborhood of the current state, and transitions between modes are difficult when the target distribution is multimodal and the modes are separated by high energy barriers. As a result, the transition strongly depends on the initial state, and thus the mixing time may grow dramatically. Therefore, when an RBM is trained on data distributions with such a structure, the mixing time of BGS on the RBM gradually increases, degrading the sampling quality. Since the degradation of the sampling quality is attributed to the locality of BGS, an additional transition kernel capable of global exploration is required to achieve rapid mixing during RBM training.

A class of nonlocal transition kernels can be constructed using a sequence of tempered distributions with gradually increasing temperatures. This class includes parallel tempering [14, 15], simulated tempering [16], and tempered transition [17, 18]. The temperature parameter controls the flatness of the target distribution, and higher-temperature distributions can generally be explored more rapidly, even using local transition kernels. Parallel tempering and simulated tempering combine two types of updates: local transitions within each tempered distribution and moves between distributions along the temperature ladder. By traversing the sequence of tempered distributions, these methods can facilitate nonlocal exploration. These methods leave the entire sequence, including the target distribution, invariant. Parallel tempering has been applied to the RBM training and has been shown to yield more stable training than methods based on BGS [19, 20]. Deep tempering (DT) has also been proposed as a learning method based on parallel tempering [21]. Whereas parallel tempering constructs a sequence of distributions by varying the temperature, DT represents the distributions in the sequence using learnable models and obtains the sequence by jointly training these models with the training RBM. DT has been shown to achieve learning performance comparable to that of parallel tempering using a smaller number of intermediate distributions [21]. In parallel tempering and DT, transitions between distributions are restricted to adjacent distributions in the temperature or model sequence. In simulated tempering, transitions between distributions are also typically implemented only between adjacent temperatures. Consequently, moving a state from a rapidly mixing high-temperature distribution to the target distribution requires multiple transitions. It is therefore algorithmically difficult for these methods to produce a nonlocal state-space move within a single transition. In contrast, tempered transition is a Metropolis–Hastings algorithm whose proposal is generated by a round-trip transition from the target distribution to the highest-temperature distribution and back. By passing through high-temperature distributions, it can readily cross energy barriers and thus has the potential to produce a nonlocal move within a single transition. However, when such a proposal is rejected, the chain remains at its current state and fails to make even a local move. Therefore, tempered transition is designed to achieve a sufficiently high acceptance probability. Although this can be achieved using a finer temperature schedule, doing so increases the computational cost. Based on the foregoing, we aim to construct a transition kernel that preserves an invariant distribution over a sequence of distributions, as do parallel tempering, DT, and simulated tempering, while retaining the potential of tempered transition to perform nonlocal moves within a single transition.

In this paper, we propose a novel transition kernel defined on the sequence of RBMs used in DT. The proposed transition kernel has a round-trip structure analogous to that of tempered transitions and thus could perform a nonlocal state-space move within a single transition. However, unlike tempered transitions, the proposed kernel does not rely on the Metropolis–Hastings algorithm and therefore does not suffer from the drawback of a nonlocal proposal being rejected, causing the chain to remain at the current state. We further ensure that the proposed transition kernel leaves the RBM sequence invariant, as does DT. We numerically demonstrate that the proposed kernel produces nonlocal moves more frequently than BGS and DT. As a result, it maintains high sampling quality even with a relatively small number of transition steps. We show

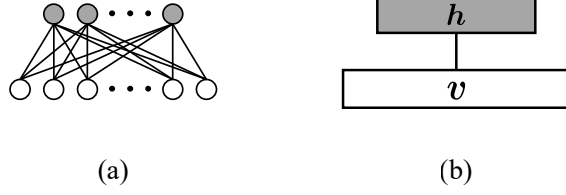


Figure 1: Schematic illustrations of an RBM: (a) a two-layer architecture with a visible layer at the bottom and a hidden layer at the top, and (b) its simplified representation.

that the proposed method achieves high learning performance. In particular, the proposed method mitigates learning failures in settings where learning based on BGS and DT typically fails.

The remainder of this paper is organized as follows. In Section 2, we introduce the RBM and its learning procedure, and numerically investigate training failures and their underlying causes. In Sections 3 and 4, we introduce DT and the proposed transition kernel with nonlocal moves, respectively. In Section 5, we evaluate the sampling and learning performance of the proposed method through numerical experiments on some synthetic and real-world datasets. Finally, in Section 6, we summarize our findings and discuss potential directions for future work.

2 Restricted Boltzmann Machines and Their Learning

2.1 Restricted Boltzmann Machines

An RBM is a Markov random field defined on a complete bipartite graph [22, 12]. The graph structure can be regarded as a two-layer architecture consisting of a visible layer, to which the visible variables are assigned, and a hidden layer, to which the hidden variables are assigned. Figure 1 illustrates the RBM structure. Consider an RBM with n visible variables $\mathbf{v} \in \{0, 1\}^n$ and m hidden variables $\mathbf{h} \in \{0, 1\}^m$. The distribution of the RBM is defined as

$$P_{\theta}(\mathbf{v}, \mathbf{h}) := \frac{1}{Z_{\theta}} \exp[-E(\mathbf{v}, \mathbf{h}; \theta)], \quad (1)$$

where E denotes the energy function defined by

$$E(\mathbf{v}, \mathbf{h}; \theta) := -\mathbf{b}^{\top} \mathbf{v} - \mathbf{c}^{\top} \mathbf{h} - \mathbf{v}^{\top} \mathbf{W} \mathbf{h}, \quad (2)$$

and Z_{θ} denotes the normalizing constant (or the partition function) expressed as

$$Z_{\theta} := \sum_{\mathbf{v}} \sum_{\mathbf{h}} \exp[-E(\mathbf{v}, \mathbf{h}; \theta)],$$

where $\sum_{\mathbf{v}}$ and $\sum_{\mathbf{h}}$ denote multiple summations over all possible realizations of the visible and hidden variables, respectively. Here, $\mathbf{b} \in \mathbb{R}^n$ and $\mathbf{c} \in \mathbb{R}^m$ denote the bias parameters for the visible and hidden variables, respectively, and $\mathbf{W} \in \mathbb{R}^{n \times m}$ denotes the connection parameters between the visible and hidden variables. These are learnable parameters and are collectively denoted by

$$\theta := \begin{pmatrix} \mathbf{b} \\ \mathbf{c} \\ \text{vec}(\mathbf{W}) \end{pmatrix} \in \mathbb{R}^{n+m+nm},$$

where $\text{vec}(\mathbf{W})$ denotes the column vector obtained by arranging the elements of $\mathbf{W}$ according to a predefined order. The visible variables $\mathbf{v}$ correspond to a data point, and the number of the visible variables, n , is thus equal to the data dimension. By contrast, the hidden variables $\mathbf{h}$ are model-internal latent variables that do not directly correspond to a data point. The number of hidden variables, m , controls the representational power of the RBM, and increasing the number of hidden variables enhances this power [23]. The conditional distributions for the visible and hidden variables are written, respectively, as

$$P_\theta(\mathbf{v} \mid \mathbf{h}) = \prod_{i=1}^n \text{Bern}(v_i \mid \text{sig}(\eta_i(\mathbf{h}; \theta))), \quad (3)$$

$$P_\theta(\mathbf{h} \mid \mathbf{v}) = \prod_{j=1}^m \text{Bern}(h_j \mid \text{sig}(\zeta_j(\mathbf{v}; \theta))), \quad (4)$$

where $\text{Bern}(x \mid p) = p^x(1-p)^{1-x}$ is a Bernoulli distribution with the probability $p \geq 0$ that the random variable x takes the value one. Here, $\text{sig}(x) := 1/(1 + e^{-x})$ is the sigmoid function, and

$$\begin{aligned} \eta_i(\mathbf{h}; \theta) &:= b_i + \sum_{j=1}^m W_{i,j} h_j, \\ \zeta_j(\mathbf{v}; \theta) &:= c_j + \sum_{i=1}^n W_{i,j} v_i. \end{aligned}$$

The conditional distributions in Eqs. (3) and (4) show that the variables in one layer are conditionally independent given the variables in the other layer. This conditional independence facilitates BGS between the two layers.

Given a current joint state $(\mathbf{v}, \mathbf{h})$, one BGS sweep first samples a hidden state from $P_\theta(\mathbf{h}' \mid \mathbf{v})$ and subsequently samples $\mathbf{v}'$ from $P_\theta(\mathbf{v}' \mid \mathbf{h}')$ given the generated hidden state. The resulting transition kernel on the joint state space is

$$K_\theta(\mathbf{v}', \mathbf{h}' \mid \mathbf{v}, \mathbf{h}) := P_\theta(\mathbf{h}' \mid \mathbf{v}) P_\theta(\mathbf{v}' \mid \mathbf{h}').$$

Although its right-hand side does not depend on the current hidden state $\mathbf{h}$, we retain $\mathbf{h}$ in the conditioning argument so that K_θ can be regarded as a Markov transition kernel on the joint state space of $\mathbf{v}$ and $\mathbf{h}$. The kernel K_θ leaves the RBM joint distribution in Eq. (1) invariant because it satisfies the balance condition expressed as

$$\sum_{\mathbf{v}} \sum_{\mathbf{h}} K_\theta(\mathbf{v}', \mathbf{h}' \mid \mathbf{v}, \mathbf{h}) P_\theta(\mathbf{v}, \mathbf{h}) = P_\theta(\mathbf{v}', \mathbf{h}').$$

When only the visible variables are retained as the Markov-chain state, the induced transition kernel on the visible state space is

$$\begin{aligned} T_\theta^{\text{BGS}}(\mathbf{v}' \mid \mathbf{v}) &:= \sum_{\mathbf{h}'} K_\theta(\mathbf{v}', \mathbf{h}' \mid \mathbf{v}, \mathbf{h}) \\ &= \sum_{\mathbf{h}'} P_\theta(\mathbf{v}' \mid \mathbf{h}') P_\theta(\mathbf{h}' \mid \mathbf{v}). \end{aligned} \quad (5)$$

Here, $\mathbf{h}'$ serves as an auxiliary variable for the visible-state Markov chain. The kernel T_θ^{BGS} leaves the marginal visible distribution in Eq. (8) invariant because it satisfies the detailed balance condition written by

$$T_\theta^{\text{BGS}}(\mathbf{v}' \mid \mathbf{v}) P_\theta(\mathbf{v}) = T_\theta^{\text{BGS}}(\mathbf{v} \mid \mathbf{v}') P_\theta(\mathbf{v}').$$

As described in Appendix B, the relaxation time, which characterizes the timescale over which the influence of the initial state decays, is determined from the second-largest eigenvalue of the transition probability matrix. For a finite Markov chain satisfying the detailed balance condition, the relaxation time appears in an upper bound on the mixing time [24]. Thus, the second-largest eigenvalue is an important measure for evaluating the sampling performance of BGS in Eq. (5).

2.2 Training Restricted Boltzmann Machines

Suppose that we obtain a dataset consisting of N_d data points corresponding to $\mathbf{v}$ expressed as

$$\mathcal{D} := \{\mathbf{d}^{(\mu)} \in \{0, 1\}^n \mid \mu = 1, 2, \dots, N_d\}$$

For the dataset $\mathcal{D}$, we consider the data distribution expressed as

$$Q_{\mathcal{D}}(\mathbf{v}) = \frac{1}{N_d} \sum_{\mu=1}^{N_d} \delta(\mathbf{v}, \mathbf{d}^{(\mu)}), \quad (6)$$

where $\delta(\cdot)$ denotes the Kronecker delta function. The log-likelihood function is then defined as

$$\ell(\theta) := \mathbb{E}_{Q_{\mathcal{D}}(\mathbf{v})}[\ln P_{\theta}(\mathbf{v})] = \frac{1}{N_d} \sum_{\mu=1}^{N_d} \ln P_{\theta}^{(\mathbf{v})}(\mathbf{d}^{(\mu)}), \quad (7)$$

where $\mathbb{E}_{Q_{\mathcal{D}}(\mathbf{v})}[\cdot]$ denotes the expectation over the data distribution, and $P_{\theta}^{(\mathbf{v})}(\mathbf{v})$ is the marginal distribution for the visible variables represented as

$$P_{\theta}^{(\mathbf{v})}(\mathbf{v}) = \frac{1}{Z_{\theta}} \exp \left[-\hat{E}(\mathbf{v}; \theta) \right], \quad (8)$$

where $\hat{E}$ denotes the energy function for the visible distribution defined by

$$\hat{E}(\mathbf{v}; \theta) := -\mathbf{b}^{\top} \mathbf{v} - \sum_{j=1}^m \text{sfp}(\zeta_j(\mathbf{v}; \theta)), \quad (9)$$

where $\text{sfp}(x) := \ln(1 + e^x)$ is the softplus function. RBM training is performed by maximizing the log-likelihood in Eq. (7) with respect to the learning parameters θ . The maximization is typically performed using gradient ascent, which iteratively updates the parameters according to

$$\theta^{\text{new}} \leftarrow \theta^{\text{old}} + \varepsilon \nabla_{\theta} \ell(\theta)|_{\theta=\theta^{\text{old}}},$$

where $\varepsilon > 0$ denotes a small positive learning rate, and θ^{old} and θ^{new} denote the values of θ before and after the update, respectively. The parameter gradient, $\nabla_{\theta} \ell$, is written as

$$\nabla_{\theta} \ell(\theta) = -\mathbb{E}_{Q_{\mathcal{D}}(\mathbf{v})}[\nabla_{\theta} \hat{E}(\mathbf{v}; \theta)] + \mathbb{E}_{P_{\theta}(\mathbf{v}, \mathbf{h})}[\nabla_{\theta} E(\mathbf{v}, \mathbf{h}; \theta)], \quad (10)$$

where $\mathbb{E}_{P_{\theta}(\mathbf{v}, \mathbf{h})}[\cdot \cdot \cdot]$ denotes the expectation over the RBM distribution in Eq. (1) defined by

$$\mathbb{E}_{P_{\theta}(\mathbf{v}, \mathbf{h})}[f(\mathbf{v}, \mathbf{h})] := \sum_{\mathbf{v}} \sum_{\mathbf{h}} f(\mathbf{v}, \mathbf{h}) P_{\theta}(\mathbf{v}, \mathbf{h}). \quad (11)$$

In particular, the gradients with respect to the parameters, b_i , c_j , and $W_{i,j}$, are represented as

$$\frac{\partial \ell(\theta)}{\partial b_i} = \mathbb{E}_{Q_{\mathfrak{D}}(\mathbf{v})}[v_i] - \mathbb{E}_{P_{\theta}(\mathbf{v}, \mathbf{h})}[v_i], \quad (12)$$

$$\frac{\partial \ell(\theta)}{\partial c_j} = \mathbb{E}_{Q_{\mathfrak{D}}(\mathbf{v})}[\text{sig}(\zeta_j(\mathbf{v}; \theta))] - \mathbb{E}_{P_{\theta}(\mathbf{v}, \mathbf{h})}[h_j], \quad (13)$$

$$\frac{\partial \ell(\theta)}{\partial W_{i,j}} = \mathbb{E}_{Q_{\mathfrak{D}}(\mathbf{v})}[v_i \text{sig}(\zeta_j(\mathbf{v}; \theta))] - \mathbb{E}_{P_{\theta}(\mathbf{v}, \mathbf{h})}[v_i h_j], \quad (14)$$

respectively. Because the expectation in Eq. (11) involves the multiple summations, $\sum_{\mathbf{v}}$ and $\sum_{\mathbf{h}}$, which cause the combinatorial explosion, maximizing Eq. (7) using gradient ascent is computationally difficult. Therefore, the expectation must be evaluated using an approximation method in practice. To properly maximize the log-likelihood using gradient ascent, it is important to accurately estimate the expectation appearing in the parameter gradient.

The expectation is typically evaluated using a sampling approximation based on BGS. Let us consider a sample distribution based on BGS, expressed as

$$P_{\text{BGS}}^s(\mathbf{v}, \mathbf{h}) := P_{\theta}(\mathbf{h} | \mathbf{v}) \sum_{\mathbf{v}'} T_{\theta, s}^{\text{BGS}}(\mathbf{v} | \mathbf{v}') P_0(\mathbf{v}'),$$

where P_0 and $T_{\theta, s}^{\text{BGS}}$ denote the initial distribution and the s -step transition kernel of BGS in Eq. (5), respectively. We then generate N_s sample points from the sample distribution, and the expectation in Eq. (11) is approximated by

$$\mathbb{E}_{P_{\theta}(\mathbf{v}, \mathbf{h})}[f(\mathbf{v}, \mathbf{h})] \approx \frac{1}{N_s} \sum_{\nu=1}^{N_s} f(\mathbf{v}^{(\nu)}, \mathbf{h}^{(\nu)}),$$

where $\{(\mathbf{v}^{(\nu)}, \mathbf{h}^{(\nu)}) | \nu = 1, 2, \dots, N_s\}$ is a set of generated sample points corresponding to $\mathbf{v}$ and $\mathbf{h}$. The accuracy of this approximation depends on the sampling quality, that is, how closely the sample distribution follows the target distribution. The sampling quality can be improved by using an initial distribution close to the target distribution. The CD and PCD methods [12, 13], which are well-known learning methods for RBMs, set the data distribution and the empirical distribution of sample points used to compute the parameter gradient in the previous iteration, respectively, as their initial distributions. These choices of initial distribution are expected to enable high-quality sampling even with only a few transitions. The sampling quality can also be improved by increasing the number of transitions, s . However, this increases the computational cost of evaluating the parameter gradients and is therefore undesirable.

The approximation accuracy can be further improved using spatial Monte Carlo integration (SMCI) [25, 26], which reduces the variance of the estimator by properly summing over a subset of variables. SMCI approximates the expectation using a sample average of conditional expectations; for example, the expectation of v_i is approximated by

$$\begin{aligned} \mathbb{E}_{P_{\theta}(\mathbf{v}, \mathbf{h})}[v_i] &= \sum_{\mathbf{h}} \left(\sum_{\mathbf{v}} v_i P_{\theta}(\mathbf{v} | \mathbf{h}) \right) P_{\theta}^{(h)}(\mathbf{h}) \\ &= \sum_{\mathbf{h}} \text{sig}(\eta_i(\mathbf{h}; \theta)) P_{\theta}^{(h)}(\mathbf{h}) \\ &\approx \frac{1}{N_s} \sum_{\nu=1}^{N_s} \text{sig}(\eta_i(\mathbf{h}^{(\nu)}; \theta)), \end{aligned}$$

where $P_\theta^{(h)}(\mathbf{h})$ is the marginal distribution for the hidden variables. SMCI-based learning achieved a higher log-likelihood than learning based on standard Monte Carlo integration, where the expectation is approximated by a sample average [27]. However, under some learning settings, the SMCI-based learning method exhibited unexpected behavior, in which the log-likelihood decreased substantially.

2.3 Preliminary Investigation of Learning Failure

In this section, we numerically identify the learning settings in which SMCI-based learning fails and investigate the cause of this failure. The training dataset was generated from a generative RBM with $n = 10$ visible and $m = 100$ hidden variables. Its parameters were independently drawn from a normal distribution whose mean and variance were zero and 0.01, respectively. We then generated $N_d = 100$ training data points from the generative RBM by running BGS, where the initial distribution was set to the uniform distribution and the number of steps was 5000. We referred to this dataset as the *GenRBM* dataset. Next, we constructed a training RBM with $n = 10$ visible and $m = 100$ hidden variables, with the bias and connection parameters were initialized to zero and using Gaussian-type Xavier initialization [28], respectively. The training RBM was then trained using a sampling approximation with a PCD-like persistent chain and semi-second-order SMCI (see Appendix A). Here, the number of steps and the sample size were set to $s = 1$ and $N_s = 100$, respectively. Gradient ascent was performed using AdaMax [29], with the learning rate fixed at 0.01 and the other hyperparameters set to their default values.

We monitored the learning behavior using three quantities: the log-likelihood in Eq. (7), the mean absolute errors (MAEs) for the expectation approximation during the learning, and the second-largest eigenvalue of the BGS transition matrix. A higher log-likelihood indicates better learning performance with less influence from approximating the expectations. Since the number of visible variables n is small in this experiment setting, the probability value for the visible distribution $P_\theta^{(v)}(\mathbf{v})$ can be exactly computed without using any approximation; thus, the log-likelihood can be evaluated. Furthermore, the expectation over the visible distribution $P_\theta^{(v)}(\mathbf{v})$ can also be exactly computed, and the expectations evaluated during training, $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i]$, $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[h_j]$, and $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i h_j]$, can thus be computed by rewriting $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i]$ and $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[h_j]$ as

$$\begin{aligned}\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[h_j] &= \mathbb{E}_{P_\theta^{(v)}(\mathbf{v})}[\text{sig}(\zeta_j(\mathbf{v}; \theta))], \\ \mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i h_j] &= \mathbb{E}_{P_\theta^{(v)}(\mathbf{v})}[v_i \text{sig}(\zeta_j(\mathbf{v}; \theta))],\end{aligned}$$

respectively. Let $\mathcal{M}_i$, $\mathcal{M}_j$, and $\mathcal{M}_{i,j}$ denote the estimators used during learning for these three expectations, and their MAEs are then written as

$$\begin{aligned}\text{MAE}_b &:= \frac{1}{n} \sum_{i=1}^n |\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i] - \mathcal{M}_i|, \\ \text{MAE}_c &:= \frac{1}{m} \sum_{j=1}^m |\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[h_j] - \mathcal{M}_j|, \\ \text{MAE}_W &:= \frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m |\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i h_j] - \mathcal{M}_{i,j}|.\end{aligned}$$

The MAEs directly measure the approximation accuracy of the expectations during learning. Larger MAEs indicate less accurate approximations. When the second-largest eigenvalue λ_2 is close to one, the relaxation time is long. Therefore, a large λ_2 means that BGS strongly depends on the initial state and requires a large number of steps to obtain high-quality samples. The method used to evaluate λ_2 used in this experiment is described in Appendix B.

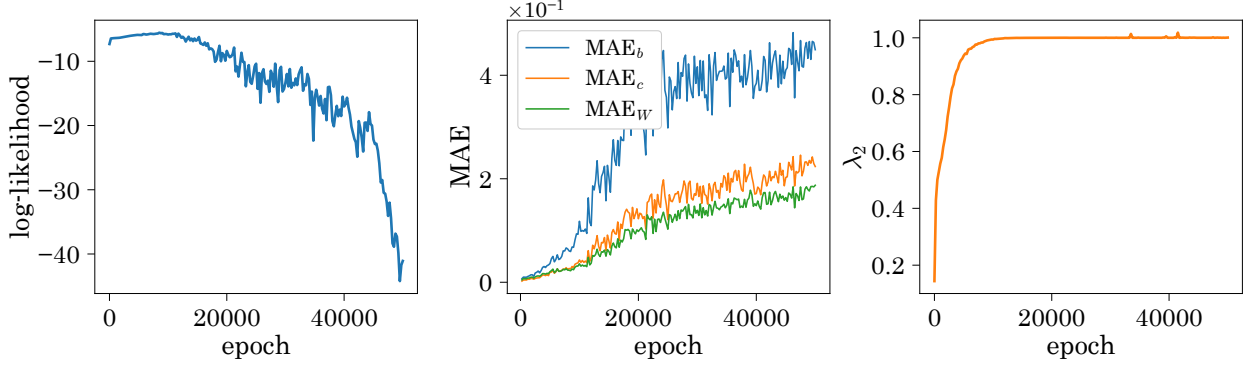


Figure 2: Log-likelihood, the MAEs, and the second-largest eigenvalue λ_2 versus the training epoch for the GenRBM dataset. The results represent averages over 10 experiments.

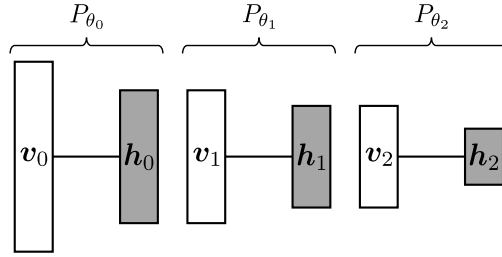


Figure 3: Illustration of the RBM sequence with $L = 2$ used in DT. The bottommost RBM corresponds to the training RBM, and the other RBMs are used for sampling.

Figure 2 shows three quantities characterizing the learning behavior as functions of the training epoch. As training progressed, the MAEs increased, implying that the parameter gradients were estimated less accurately. This deterioration in gradient estimation, in turn, led to a decrease in the log-likelihood. Moreover, the estimator of λ_2 converged to one as the training began to fail. As the relaxation time increased, the sampling became more strongly dependent on the initial states, making it more difficult to sample from the target distribution within a limited number of sampling steps. These results suggest that the resulting degradation in the sampling quality of BGS caused the learning process to fail. BGS is a local transition kernel, and increasing the relaxation time implies that its locality becomes stronger. Thus, the learning failure could be mitigated by introducing transition kernels that enable nonlocal transitions in the visible state space.

3 Deep Tempering

DT [21] is a global transition kernel based on parallel tempering [14, 15] for RBMs. A sequence of distributions for parallel tempering is obtained by stacking auxiliary RBMs and learning them jointly with the training RBM. The training RBM is denoted by $P_{\theta_0}(v_0, h_0)$, and we consider L additional RBMs expressed as

$$P_{\theta_\ell}(v_\ell, h_\ell). \quad (\ell = 1, 2, \dots, L)$$

Here, θ_ℓ denotes the parameter of the ℓ -th RBM, and $v_\ell \in \{0, 1\}^{n_\ell}$ and $h_\ell \in \{0, 1\}^{m_\ell}$ are the visible and hidden variables for ℓ -th RBM. The parameters are collectively denoted by $\Theta := (\theta_0, \theta_1, \dots, \theta_L)$. Here, the number of visible variables, respectively, of the $(\ell + 1)$ -th RBM is constrained to equal the number of

Algorithm 1 Swap kernel

```

1: function SWAPKERNEL( $\ell, \mathbf{h}_\ell, \mathbf{v}_{\ell+1}$ )
2:   Sample  $u \sim \text{Unif}(0, 1)$ 
3:   if  $u \leq a_{\ell, \ell+1}(\mathbf{h}_\ell, \mathbf{v}_{\ell+1})$  then
4:      $(\mathbf{h}'_\ell, \mathbf{v}'_{\ell+1}) \leftarrow (\mathbf{v}_{\ell+1}, \mathbf{h}_\ell)$ 
5:   else
6:      $(\mathbf{h}'_\ell, \mathbf{v}'_{\ell+1}) \leftarrow (\mathbf{h}_\ell, \mathbf{v}_{\ell+1})$ 
7:   end if
8:   return  $(\mathbf{h}'_\ell, \mathbf{v}'_{\ell+1})$ 
9: end function

```

hidden variables in the ℓ -th RBM, i.e., $n_{\ell+1} = m_\ell$ for $\ell = 0, 1, \dots, L-1$. Figure 3 illustrates the sequence of the training RBM and the additional RBMs. The training and additional RBMs are trained by iteratively updating the parameters as

$$\Theta^{(t+1)} \leftarrow \Theta^{(t)} + \varepsilon \nabla_{\Theta'} \bar{\ell}(\Theta' \mid \Theta^{(t)})|_{\Theta'=\Theta^{(t)}}, \quad (15)$$

using the function defined by

$$\bar{\ell}(\Theta' \mid \Theta) := \ell(\theta'_0) + \sum_{\ell=1}^L \mathbb{E}_{Q_{\mathfrak{D}}^{(\ell)}(\mathbf{v}_\ell \mid \Theta)} [\ln P_{\theta'_\ell}(\mathbf{v}_\ell)], \quad (16)$$

where $Q_{\mathfrak{D}}^{(\ell)}$ denotes the extended data distribution represented as

$$Q_{\mathfrak{D}}^{(\ell)}(\mathbf{v}_\ell \mid \Theta) := \sum_{\mathbf{v}_{\ell-1}} \sum_{\mathbf{h}_{\ell-1}} \delta(\mathbf{v}_\ell, \mathbf{h}_{\ell-1}) P_{\theta_{\ell-1}}(\mathbf{h}_{\ell-1} \mid \mathbf{v}_{\ell-1}) Q_{\mathfrak{D}}^{(\ell)}(\mathbf{v}_{\ell-1} \mid \Theta), \quad (17)$$

where $Q_{\mathfrak{D}}^{(0)} = Q_{\mathfrak{D}}$. The gradient with respect to the ℓ -th parameter θ_ℓ is written as

$$\nabla_{\theta'_\ell} \bar{\ell}(\Theta' \mid \Theta) = -\mathbb{E}_{Q_{\mathfrak{D}}^{(\ell)}(\mathbf{v}_\ell \mid \Theta)} [\nabla_{\theta'_\ell} \hat{E}(\mathbf{v}_\ell; \theta'_\ell)] + \mathbb{E}_{P_{\theta_\ell}(\mathbf{v}_\ell, \mathbf{h}_\ell)} [\nabla_{\theta'_\ell} E(\mathbf{v}_\ell, \mathbf{h}_\ell; \theta'_\ell)]. \quad (18)$$

The first term can be readily estimated using a sampling approximation based on ancestral sampling from the extended data distribution in Eq. (17). In contrast, the second term is estimated using a sampling approximation based on parallel tempering on the joint distribution $P_{\Theta}(\mathbf{V}) := \prod_{\ell=0}^L P_{\theta_\ell}^{(v)}(\mathbf{v}_\ell)$, where $\mathbf{V} := (\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_L)$.

DT considers the swap probability between the ℓ -th hidden variables and the $(\ell+1)$ -th visible variables, defined by

$$a_{\ell, \ell+1}(\mathbf{h}_\ell, \mathbf{v}_{\ell+1}) := \min \left(1, \frac{P_{\theta_{\ell+1}}^{(v)}(\mathbf{h}_\ell) P_{\theta_\ell}^{(h)}(\mathbf{v}_{\ell+1})}{P_{\theta_{\ell+1}}^{(v)}(\mathbf{v}_{\ell+1}) P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell)} \right). \quad (19)$$

The corresponding swap kernel is defined by

$$S_{\ell, \ell+1}(\mathbf{h}'_\ell, \mathbf{v}'_{\ell+1} \mid \mathbf{h}_\ell, \mathbf{v}_{\ell+1}) := a_{\ell, \ell+1}(\mathbf{h}_\ell, \mathbf{v}_{\ell+1}) \delta(\mathbf{h}'_\ell, \mathbf{v}_{\ell+1}) \delta(\mathbf{v}'_{\ell+1}, \mathbf{h}_\ell) \\ + [1 - a_{\ell, \ell+1}(\mathbf{h}_\ell, \mathbf{v}_{\ell+1})] \delta(\mathbf{v}'_{\ell+1}, \mathbf{v}_{\ell+1}) \delta(\mathbf{h}'_\ell, \mathbf{h}_\ell), \quad (20)$$

Algorithm 2 One-step transition of deep tempering

Input: Current visible states $\mathbf{V} = (\mathbf{v}_0, \dots, \mathbf{v}_L)$ and parity indicator $c \in \{0, 1\}$

Output: Updated visible states $\mathbf{V}' = (\mathbf{v}'_0, \dots, \mathbf{v}'_L)$ and parity indicator c'

```
1:  $\tilde{\mathbf{V}} \leftarrow \mathbf{V}, \quad \mathcal{I}_c \leftarrow \{\ell \in \{0, \dots, L-1\} \mid \ell \equiv c \pmod{2}\}$ 
2: for all  $\ell \in \mathcal{I}_c$  do ▷ Visible-space swap
3:   Sample  $\mathbf{h}_\ell \sim P_{\theta_\ell}(\mathbf{h}_\ell \mid \tilde{\mathbf{v}}_\ell)$ 
4:    $(\tilde{\mathbf{h}}_\ell, \tilde{\mathbf{v}}_{\ell+1}) \leftarrow \text{SWAPKERNEL}(\ell, \mathbf{h}_\ell, \tilde{\mathbf{v}}_{\ell+1})$ 
5:   Sample  $\tilde{\mathbf{v}}_\ell \sim P_{\theta_\ell}(\mathbf{v}_\ell \mid \tilde{\mathbf{h}}_\ell)$ 
6: end for
7: for all  $\ell \in \{0, \dots, L\}$  do ▷ One-step BGS
8:   Sample  $\mathbf{h}'_\ell \sim P_{\theta_\ell}(\mathbf{h}_\ell \mid \tilde{\mathbf{v}}_\ell)$  and  $\mathbf{v}'_\ell \sim P_{\theta_\ell}(\mathbf{v}_\ell \mid \mathbf{h}'_\ell)$ 
9: end for
10:  $c' \leftarrow 1 - c$ 
11: return  $(\mathbf{V}', c')$ 
```

and satisfies the detailed balance condition with respect to $P_{\theta_{\ell+1}}^{(v)}(\mathbf{v}_{\ell+1})P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell)$. The implementation of the swap kernel is described by Algorithm 1. Using the swap kernel, we define a visible-space swap kernel as

$$\bar{K}_{\ell, \ell+1}(\mathbf{v}'_\ell, \mathbf{v}'_{\ell+1} \mid \mathbf{v}_\ell, \mathbf{v}_{\ell+1}) := \sum_{\mathbf{h}_\ell} \sum_{\mathbf{h}'_\ell} P_{\theta_\ell}(\mathbf{v}'_\ell \mid \mathbf{h}'_\ell) S_{\ell, \ell+1}(\mathbf{h}'_\ell, \mathbf{v}'_{\ell+1} \mid \mathbf{h}_\ell, \mathbf{v}_{\ell+1}) P_{\theta_\ell}(\mathbf{h}_\ell \mid \mathbf{v}_\ell). \quad (21)$$

This satisfies the balance condition for $P_{\theta_\ell}^{(v)}(\mathbf{v}_\ell)P_{\theta_{\ell+1}}^{(v)}(\mathbf{v}_{\ell+1})$. DT performs parallel tempering using the visible-space swap kernel in Eq. (21) and BGS on each RBM (see Algorithm 2). The corresponding transition kernel is denoted by $T_{\Theta}^{\text{DT}}(\mathbf{V}' \mid \mathbf{V})$. For simplicity, the parity indicator is omitted from the notation. At each transition, the parity indicator is passed to the next transition and is deterministically updated according to $c' = 1 - c$. The transition kernel satisfies the balance condition with respect to $P_{\Theta}(\mathbf{V})$; that is, it leaves $P_{\Theta}(\mathbf{V})$ invariant [21]. We consider a sample distribution based on DT expressed as

$$P_{\text{DT}}^s(\mathbf{V}, \mathbf{H}) := \prod_{\ell=0}^L P_{\theta_\ell}(\mathbf{h}_\ell \mid \mathbf{v}_\ell) \sum_{\mathbf{V}'} T_{\Theta, s}^{\text{DT}}(\mathbf{V} \mid \mathbf{V}') P_0(\mathbf{V}'),$$

where P_0 and $T_{\Theta, s}^{\text{DT}}$ denote the initial distribution and the s -step transition kernel of DT, respectively. Here, $\mathbf{H} := (\mathbf{h}_0, \mathbf{h}_1, \dots, \mathbf{h}_L)$. Using sample points generated from the sample distribution, the expectation in the second term of the parameter gradient in Eq. (18) is approximated by a sample average.

In standard parallel tempering, a sequence of intermediate distributions is constructed by varying an inverse-temperature ladder. In contrast, DT constructs the sequence through learning. This learning-based construction can reduce the number of intermediate distributions required for effective sampling. Indeed, DT has been shown to achieve competitive or better empirical performance than standard parallel tempering while using fewer intermediate distributions [21]. Moreover, because higher-layer RBMs typically have fewer hidden units than lower-layer RBMs, their state spaces are smaller, which is expected to facilitate mixing. However, the swap mechanism, which plays a key role in enabling a nonlocal transition in the visible state space of a lower RBM, is local with respect to the RBM index ℓ , and a single transition based on DT can only moves a state by at most one layer. Therefore, for a state at the ℓ' -th RBM to reach the ℓ -th RBM, where $\ell' > \ell$, at least $(\ell' - \ell)$ DT transitions are required. Thus, although DT can in principle achieve a nonlocal transition in the visible state space of the training RBM, such a transition necessarily requires multiple DT transitions. To address this limitation, we propose a transition kernel that can transfer a state directly between nonadjacent RBMs in a single transition. The proposed kernel thereby enables a nonlocal transition within one step while leaving the target distribution invariant.

4 Proposed Non-local Transition Kernel

The proposed transition kernel is formulated recursively and has a round-trip structure: the states are first propagated upward through the RBM sequence, one BGS transition is performed at the topmost RBM, and the resulting state is subsequently propagated downward. For the topmost RBM, we use the one-step BGS kernel as the base transition kernel K_L , i.e., $K_L(\mathbf{v}'_L | \mathbf{v}_L) := T_{\theta_L}^{\text{BGS}}(\mathbf{v}'_L | \mathbf{v}_L)$. We next consider the topmost RBM and the RBM immediately below it. Starting from the current state $(\mathbf{v}_{L-1}, \mathbf{v}_L)$, we first sample $\mathbf{h}_{L-1} \sim P_{\theta_{L-1}}(\mathbf{h}_{L-1} | \mathbf{v}_{L-1})$. The sampled state $\mathbf{h}_{L-1}$ and the current state $\mathbf{v}_L$ are then updated using the swap kernel $S_{L-1,L}$, yielding the post-swap states $(\mathbf{h}_{L-1}^+, \mathbf{v}_L^+)$. The kernel T_L is subsequently applied to $\mathbf{v}_L^+$, producing the intermediate state $\hat{\mathbf{v}}_L$. During the downward part of the transition, $\mathbf{h}_{L-1}^+$ and $\hat{\mathbf{v}}_L$ are updated again using $S_{L-1,L}$, yielding $(\mathbf{h}_{L-1}^-, \mathbf{v}'_L)$. Finally, $\mathbf{v}'_{L-1}$ is sampled from $P_{\theta_{L-1}}(\mathbf{v}_{L-1} | \mathbf{h}_{L-1}^-)$. The corresponding transition kernel is defined by

$$\begin{aligned} T_{L-1}(\mathbf{v}'_{L-1}, \mathbf{v}'_L | \mathbf{v}_{L-1}, \mathbf{v}_L) := & \sum_{\mathbf{h}_{L-1}^-, \mathbf{h}_{L-1}^+, \hat{\mathbf{v}}_L, \mathbf{v}_L^+, \mathbf{h}_{L-1}} P_{\theta_{L-1}}(\mathbf{v}'_{L-1} | \mathbf{h}_{L-1}^-) \\ & \times S_{L-1,L}(\mathbf{h}_{L-1}^-, \mathbf{v}'_L | \mathbf{h}_{L-1}^+, \hat{\mathbf{v}}_L) T_L(\hat{\mathbf{v}}_L | \mathbf{v}_L^+) \\ & \times S_{L-1,L}(\mathbf{h}_{L-1}^+, \mathbf{v}_L^+ | \mathbf{h}_{L-1}^-, \mathbf{v}_L) P_{\theta_{L-1}}(\mathbf{h}_{L-1} | \mathbf{v}_{L-1}). \end{aligned} \quad (22)$$

We next consider a more general transition kernel defined on the visible variables from the ℓ -th RBM to the topmost RBM. We denote the corresponding collection of visible variables by $\mathbf{V}_{\ell:L} := (\mathbf{v}_\ell, \mathbf{v}_{\ell+1}, \dots, \mathbf{v}_L)$. Given the state $\mathbf{V}_{\ell:L}$, we first sample $\mathbf{h}_\ell \sim P_{\theta_\ell}(\mathbf{h}_\ell | \mathbf{v}_\ell)$. The sampled state $\mathbf{h}_\ell$ and the current state $\mathbf{v}_{\ell+1}$ are updated using $S_{\ell,\ell+1}$, producing $(\mathbf{h}_\ell^+, \mathbf{v}_{\ell+1}^+)$. The recursively defined kernel $T_{\ell+1}$ is then applied to $(\mathbf{v}_{\ell+1}^+, \mathbf{V}_{\ell+2:L})$, yielding $(\hat{\mathbf{v}}_{\ell+1}, \mathbf{V}'_{\ell+2:L})$. Here, $\hat{\mathbf{v}}_{\ell+1}$ denotes the intermediate visible state returned to the $(\ell+1)$ -th RBM after completing the round trip through the upper sequence. During the downward transition, $\mathbf{h}_\ell^+$ and $\hat{\mathbf{v}}_{\ell+1}$ are updated using $S_{\ell,\ell+1}$, producing $(\mathbf{h}_\ell^-, \mathbf{v}'_{\ell+1})$. Finally, $\mathbf{v}'_\ell$ is sampled from $P_{\theta_\ell}(\mathbf{v}_\ell | \mathbf{h}_\ell^-)$. Accordingly, for $\ell = 0, 1, \dots, L-1$, the recursive transition kernel is defined as

$$\begin{aligned} T_\ell(\mathbf{V}'_{\ell:L} | \mathbf{V}_{\ell:L}) := & \sum_{\mathbf{h}_\ell^-, \mathbf{h}_\ell^+, \hat{\mathbf{v}}_{\ell+1}, \mathbf{v}'_{\ell+1}, \mathbf{h}_\ell} P_{\theta_\ell}(\mathbf{v}'_\ell | \mathbf{h}_\ell^-) \\ & \times S_{\ell,\ell+1}(\mathbf{h}_\ell^-, \mathbf{v}'_{\ell+1} | \mathbf{h}_\ell^+, \hat{\mathbf{v}}_{\ell+1}) \\ & \times T_{\ell+1}(\hat{\mathbf{v}}_{\ell+1}, \mathbf{V}'_{\ell+2:L} | \mathbf{v}_{\ell+1}^+, \mathbf{V}_{\ell+2:L}) \\ & \times S_{\ell,\ell+1}(\mathbf{h}_\ell^+, \mathbf{v}_{\ell+1}^+ | \mathbf{h}_\ell^-, \mathbf{v}_{\ell+1}) P_{\theta_\ell}(\mathbf{h}_\ell | \mathbf{v}_\ell). \end{aligned} \quad (23)$$

The proposed transition kernel for the entire RBM sequence is obtained by setting $\ell = 0$, i.e.,

$$T_\Theta^{\text{prop}}(\mathbf{V}' | \mathbf{V}) := T_0(\mathbf{V}' | \mathbf{V}).$$

Figure 4 illustrates the round-trip sampling procedure for the proposed transition kernel, and Algorithm 3 shows its implementation. The proposed transition kernel leaves $P_\Theta(\mathbf{V})$ invariant, as described in Appendix C. Learning based on the proposed transition kernel follows the same procedure as learning based on DT. Specifically, the parameter set Θ is updated by gradient ascent using the parameter gradient in Eq. (18), and the expectation appearing in the gradient is evaluated using a sampling approximation based on the proposed transition kernel.

Owing to its round-trip structure, the proposed transition kernel may transfer a state from an upper-layer RBM, where mixing is relatively easy, to the bottommost RBM (i.e., the training RBM) within a single transition. Consequently, nonlocal transitions in the visible state space of the training RBM are expected to occur within a small number of transition steps.

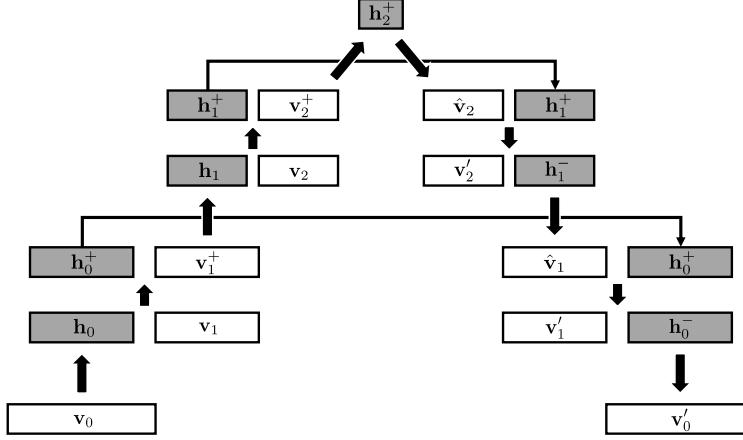


Figure 4: Illustration of the sampling procedure for the proposed transition kernel on an RBM sequence with $L = 2$. The upward pass sequentially applies the swap operations from lower to upper RBMs, followed by a one-step BGS transition at the topmost RBM, and the downward pass applies the swap operations in the reverse order of the upward pass.

Table 1: Experimental settings for training the RBMs.

Setting	GenRBM	Islands	Pentagon	Iris	Wine	Seeds
Data dimension n	10	25	25	20	65	35
Data size N_d	100	100	100	150	178	210
Number of hidden variables m	100	100	100	100	100	100
Sample size N_s	100	100	100	150	100	100
Learning rate ε	0.01	0.001	0.001	0.002	0.001	0.001

5 Numerical Experiments

In this section, we numerically evaluate the performance of learning using the proposed transition kernel and the nonlocality of its transitions.

5.1 Experimental Settings

We evaluated the learning performance on six datasets: GenRBM, islands, pentagon, iris, wine, and seeds. The RBM parameter initialization, optimization method, and sampling approximations for the first and second terms of the parameter gradient in Eq. (18) were common to all datasets. Specifically, for both the training RBM and the stacked RBMs, the bias parameters were initialized to zero, while the weight parameters were initialized using Gaussian Xavier initialization [28]. The parameters were optimized by gradient ascent using AdaMax [29], with its default hyperparameter values except for the learning rate. The expectation in the first term of the parameter gradient was approximated by drawing one sample from the extended data distribution in Eq. (17) for each data point. Thus, the total sample size was equal to the dataset size. In contrast, the expectation in the second term was approximated using PCD-like persistent chains and semi-second-order SMCI, with the number of transition steps set to $s = 1$. For DT and the proposed transition kernel, we used an RBM sequence with $L = 3$ for all datasets unless otherwise specified. The numbers of hidden variables in the RBMs stacked above the training RBM were set to $(m_1, m_2, m_3) = (50, 25, 12)$. The remaining experimental settings are described below, and the dataset-specific hyperparameters are summarized in Table 1.

Algorithm 3 One-step transition of the proposed transition kernel

Input: Current visible-state realization $\mathbf{V} = (\mathbf{v}_0, \dots, \mathbf{v}_L)$ **Output:** Updated visible-state realization $\mathbf{V}' = (\mathbf{v}'_0, \dots, \mathbf{v}'_L)$

```
1:  $\mathbf{v}_0^+ \leftarrow \mathbf{v}_0$ 
2: for  $\ell = 0, \dots, L - 1$  do ▷ Upward pass
3:   Sample  $\mathbf{h}_\ell \sim P_{\theta_\ell}(\mathbf{h}_\ell \mid \mathbf{v}_\ell^+)$ 
4:    $(\mathbf{h}_\ell^+, \mathbf{v}_{\ell+1}^+) \leftarrow \text{SWAPKERNEL}(\ell, \mathbf{h}_\ell, \mathbf{v}_{\ell+1})$ 
5: end for
6: Sample  $\mathbf{h}_L^+ \sim P_{\theta_L}(\mathbf{h}_L \mid \mathbf{v}_L^+)$  and  $\hat{\mathbf{v}}_L \sim P_{\theta_L}(\mathbf{v}_L \mid \mathbf{h}_L^+)$  ▷ One-step BGS
7: for  $\ell = L - 1, \dots, 0$  do ▷ Downward pass
8:    $(\mathbf{h}_\ell^-, \mathbf{v}_{\ell+1}') \leftarrow \text{SWAPKERNEL}(\ell, \mathbf{h}_\ell^+, \hat{\mathbf{v}}_{\ell+1})$ 
9:   Sample  $\hat{\mathbf{v}}_\ell \sim P_{\theta_\ell}(\mathbf{v}_\ell \mid \mathbf{h}_\ell^-)$ 
10: end for
11:  $\mathbf{v}'_0 \leftarrow \hat{\mathbf{v}}_0$ 
12: return  $\mathbf{V}'$ 
```

GenRBM Dataset. The GenRBM dataset used in the preliminary experiment was also used in the main experiment. For details of this dataset, see Section 2.3. For DT and the proposed transition kernel, in addition to the default setting of $L = 3$, we examined shallower RBM sequences with $L = 1$ and $L = 2$. For $L = 1, 2$, and 3 , the numbers of hidden variables in the stacked RBMs were set to $m_1 = 50$, $(m_1, m_2) = (50, 25)$, and $(m_1, m_2, m_3) = (50, 25, 12)$, respectively.

Islands Dataset. The islands dataset is generated from a two-component mixture of product Bernoulli distributions expressed as

$$P_{\text{islands}}(\mathbf{v} \mid a) := \frac{1}{2} \sum_{z \in \{-1, +1\}} \prod_{i=1}^n \text{Bern}\left(v_i \mid \frac{1}{2} + az\right), \quad (24)$$

where the parameter $a \in [0, 1/2]$ controls the separation between the two mixture components: a larger value of a yields more clearly separated modes. Figure 5 shows the visualization of sample points and a principal component analysis (PCA) projection for the datasets with $a = 0.4$. We set $a = 0.4$.

Pentagon Dataset. The pentagon dataset is generated from a five-component mixture of product Bernoulli distributions expressed as

$$P_{\text{pentagon}}(\mathbf{v} \mid \kappa) := \frac{1}{5} \sum_{z=1}^5 \prod_{i=1}^n \text{Bern}(v_i \mid \text{sig}(\kappa \rho_{z,i})),$$

where $\rho_{z,i} := \cos\left[\frac{2\pi}{5}(\chi_i - z)\right]$ and $\chi_i := \left\lfloor \frac{5i}{n} \right\rfloor$. The parameter $\kappa \geq 0$ controls the separation among the five mixture components: a larger value of κ yields more clearly separated modes. Figure 5 shows the visualization of the datasets with $\kappa = 6$. We set $\kappa = 6$.

Iris Dataset. The iris dataset is a classification dataset comprising 150 data points from three iris species: iris setosa, iris versicolor, and iris virginica. Each data point has four continuous features: sepal length, sepal width, petal length, and petal width. We transformed each continuous feature into binary variables using rank-hot encoding, also known as thermometer encoding. Specifically, each feature was divided into K_{enc}

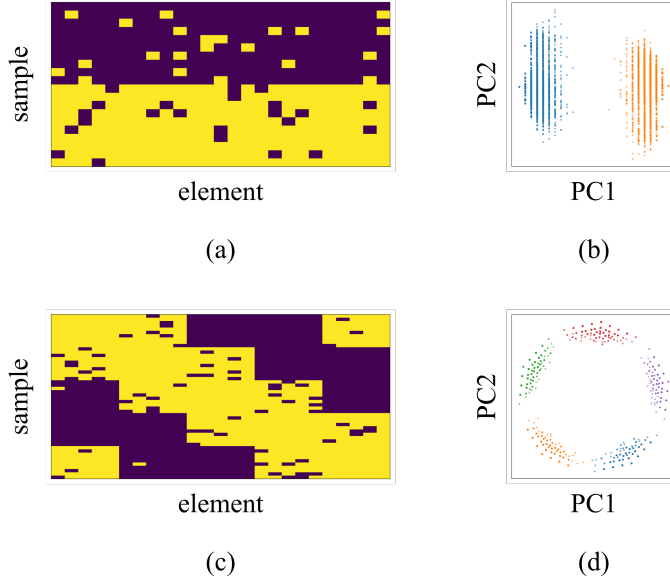


Figure 5: Visualization of low-rank synthetic datasets with data dimension $n = 25$: (a) samples and (b) a PCA projection for the islands dataset with $a = 0.4$, and (c) samples and (d) a PCA projection for the pentagon dataset with $\kappa = 6$.

bins based on its quantiles, and the bins were ordered by feature value. For $k \in 1, 2, \dots, K_{\text{enc}}$, the k -th bin was encoded as a binary vector of dimension $(K_{\text{enc}} - 1)$, whose first $(k - 1)$ elements were ones and whose remaining elements were zeros. Consequently, the four continuous features were represented by a total of $n = 4(K_{\text{enc}} - 1)$ binary variables. We set $K_{\text{enc}} = 6$, yielding $n = 20$.

Wine Dataset. The wine dataset is a classification dataset comprising 178 data points from three wine cultivars. Each data point has 13 continuous features obtained from chemical analyses of wine samples. Each continuous feature was transformed into binary variables using rank-hot encoding with $K_{\text{enc}} = 6$, in the same manner as for the iris dataset. Consequently, the total data dimension was $n = 13(K_{\text{enc}} - 1) = 65$.

Seeds Dataset. The seeds dataset is a classification dataset comprising 210 data points from three wheat varieties. Each data point has seven continuous features describing the geometrical properties of a wheat kernel. Each continuous feature was transformed into binary variables using rank-hot encoding with $K_{\text{enc}} = 6$, in the same manner as for the iris and wine datasets. Consequently, the total data dimension was $n = 7(K_{\text{enc}} - 1) = 35$.

5.2 Learning Performance

We first demonstrate that the proposed transition kernel mitigates the learning failure observed for the GenRBM dataset in Section 2.3. Figure 6 shows the log-likelihood and MAE as functions of the training epoch for BGS, DT, and the proposed transition kernel. For both DT and the proposed transition kernel, the degradation in the log-likelihood is progressively mitigated as L increases. In particular, the proposed transition kernel mitigates this degradation more effectively than DT, and learning appears stable at $L = 3$.

Next, we compare the learning performance on the five datasets with clustered structures: the islands, pentagon, iris, wine, and seeds datasets. The datasets lead to multimodal learned distributions with high energy barriers, which may make BGS transitions more local. We evaluated the learning performance using

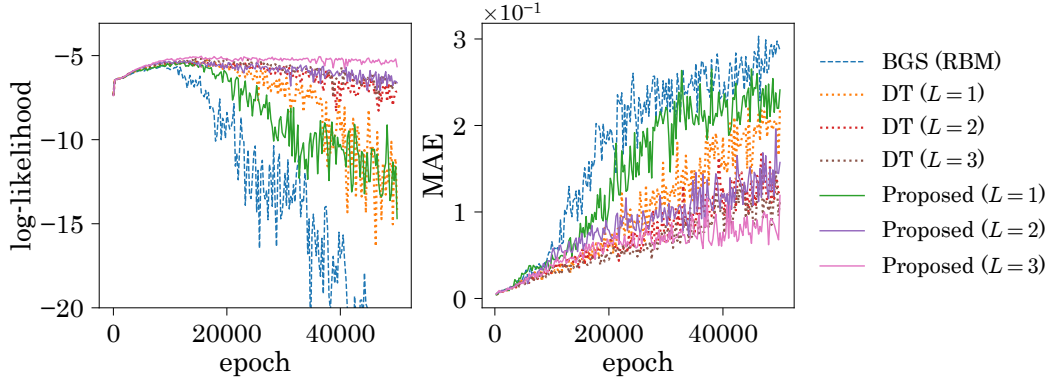


Figure 6: Log-likelihood and MAE versus the training epoch for the GenRBM dataset. The results represent averages over 10 experiments.

the log-likelihood of the training dataset. However, for the wine and seeds datasets, exact computation of the partition function was computationally intractable owing to their high dimensionality. Therefore, we approximated the log-likelihood by estimating the partition function using marginalized annealed importance sampling [30]. Figure 7 shows the log-likelihood as a function of the training epoch for the five datasets. The proposed transition kernel achieves learning performance comparable to or better than that of BGS and DT across these datasets.

5.3 Sampling Performance

We evaluated the sampling quality of the proposed transition kernel using the RBM sequence that achieved the highest log-likelihood among all RBM sequences obtained from the experimental trials. All RBM sequences were trained using the proposed transition kernel. For the GenRBM dataset, the model was selected from those trained with $L = 3$.

We visualized the transition trajectories to examine whether nonlocal transitions were achieved. The initial states were set to the states obtained from the persistent chains during training. DT and the proposed transition kernel were then applied to the RBM sequence, and BGS was applied to the bottommost training RBM. We performed 100 transition steps and plotted the resulting trajectories in the PCA space of the training dataset. Figure 8 depicts the resulting trajectories for all datasets. The trajectories generated by the proposed transition kernel covered a broader range of states than those generated by the other transition kernels. In particular, for datasets with clustered structures, the proposed transition kernel moved between clusters more frequently.

Next, we evaluated the dependence of the transition kernel on its initial state. Multiple chains were run independently from the same initial state, which was taken from a persistent chain used during training. At each transition step s , we measured the effect of the initial state using the total variation (TV) distance between the visible distribution of the trained RBM and the sample distribution for the sample points obtained after s transition steps. Let the sample distribution be denoted by Q_s . The TV distance is then defined as

$$D_{\text{TV}}(P_\theta, Q_s) := \frac{1}{2} \sum_v |P_\theta(v) - Q_s(v)|. \quad (25)$$

Because the evaluation of the TV distance requires summation over all possible visible states, it is computationally feasible only when the number of visible variables is small. We therefore used only the RBM sequences trained on four datasets: GenRBM, islands, pentagon, and iris. As the number of transition steps s increases, the empirical distribution of the samples is expected to approach the target distribution, thereby

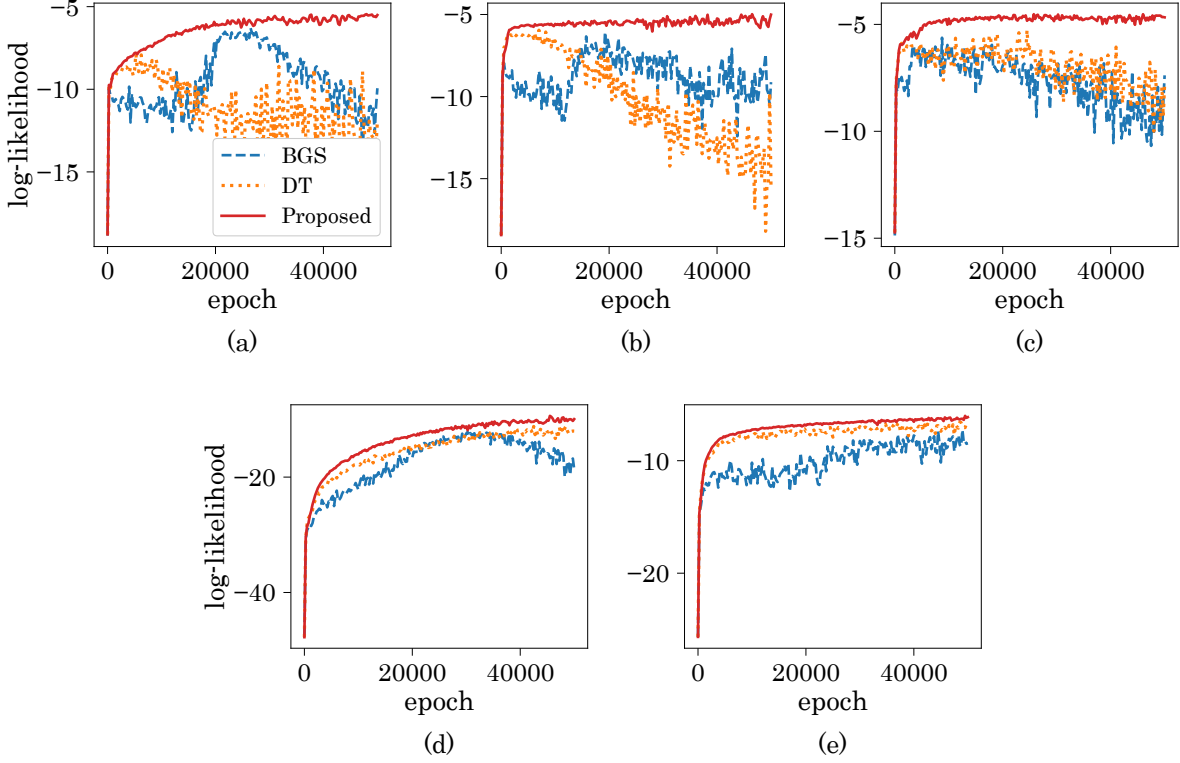


Figure 7: Log-likelihoods versus the training epoch for the (a) islands, (b) pentagon, (c) iris, (d) wine, and (e) seeds datasets. The results represent averages over 10 experiments.

reducing the TV distance. If a transition kernel depends only weakly on the initial state, the TV distance is expected to decrease rapidly. Therefore, a faster decay of the TV distance indicates weaker initial-state dependence and suggests faster mixing. However, this decay does not directly measure the mixing time because it is evaluated from a specific initial state and only in the visible-variable space of the bottommost RBM of the RBM sequence. Figure 9 depicts the TV distances for the four datasets. For all four datasets, the median TV distance obtained with the proposed transition kernel decreased more rapidly than the corresponding distances obtained with BGS and DT. The proposed kernel also attained lower TV distances within the evaluated range of transition steps. These results indicate that the proposed transition kernel reduced the dependence on the initial state more rapidly than the other kernels.

6 Conclusion

In this study, we proposed a new transition kernel defined on an RBM sequence constructed for DT. The proposed kernel performs a round trip through the RBM sequence, moving from the bottom RBM to the topmost RBM and then returning to the bottom RBM. This construction is intended to produce nonlocal transitions within a small number of transition steps by leveraging the smaller state spaces of the higher-layer RBMs. Our numerical experiments showed that the proposed transition kernel explored a broader range of states and moved between modes more frequently than BGS and DT. It also reduced the dependence on the initial state more rapidly, as indicated by the decay of the TV distance. Furthermore, learning based on the proposed transition kernel mitigated the degradation in log-likelihood observed in SMCI-based learning. These results suggest that the proposed transition kernel facilitates nonlocal transitions within a



Figure 8: Transition trajectories of BGS, DT, and the proposed kernel for the six datasets.

small number of steps. This property could improve the accuracy of the sampling approximation used for parameter-gradient estimation with a limited number of transition steps and thereby enhance the stability of RBM learning.

MCMC-based sampling approximations are generally biased when the Markov chain is terminated after a finite number of transition steps. Although this bias typically decreases as the number of steps increases, exact unbiasedness generally requires samples from the stationary distribution, which cannot generally be obtained simply by running a standard MCMC chain for a finite number of steps. This gap between the theoretical gradient and its practical approximation may affect the learning process. Unbiased MCMC estimators based on coupled Markov chains have recently been developed to remove the bias [31]. Combining such estimators with the proposed transition kernel is a possible direction for future work. Other important directions include a theoretical analysis of the mixing properties of the proposed kernel and an investigation of how the depth and layer dimensions of the RBM sequence affect its performance.

References

- [1] R. Salakhutdinov and H. Larochelle, “Efficient learning of deep boltzmann machines,” in *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 693–700, JMLR Workshop and Conference Proceedings, 2010.

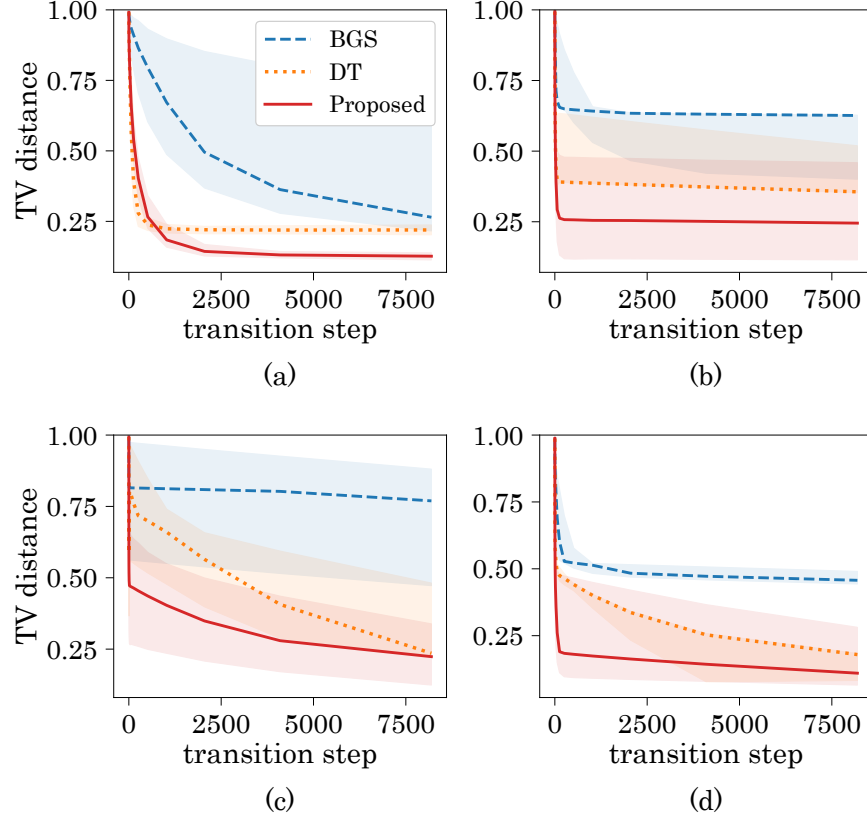


Figure 9: TV distances in Eq. (25) for the (a) GenRBM, (b) islands, (c) pentagon, and (d) iris datasets. The solid curves and shaded regions represent the medians and the 10th–90th percentile ranges, respectively.

- [2] R. Salakhutdinov, A. Mnih, and G. Hinton, “Restricted boltzmann machines for collaborative filtering,” in *Proceedings of the 24th international conference on Machine learning*, pp. 791–798, 2007.
- [3] B. Abdollahi and O. Nasraoui, “Explainable restricted boltzmann machines for collaborative filtering,” 2016.
- [4] A. Decelle, B. Seoane, and L. Rosset, “Unsupervised hierarchical clustering using the learning dynamics of restricted boltzmann machines,” *Phys. Rev. E*, vol. 108, p. 014110, Jul 2023.
- [5] A. Decelle, C. Furtlehner, A. D. J. N. Gómez, and B. Seoane, “Inferring effective couplings with restricted Boltzmann machines,” *SciPost Phys.*, vol. 16, p. 095, 2024.
- [6] U. Fiore, F. Palmieri, A. Castiglione, and A. De Santis, “Network anomaly detection with the restricted boltzmann machine,” *Neurocomputing*, vol. 122, pp. 13–23, 2013.
- [7] K. Do, T. Tran, and S. Venkatesh, “Energy-based anomaly detection for mixed data,” *Knowledge and Information Systems*, vol. 57, p. 413–435, Feb. 2018.
- [8] K. Sekimoto, C. Takahashi, and M. Yasuda, “Quasi-free energy evaluation of gaussian-bernoulli restricted boltzmann machine for anomaly detection,” *Nonlinear Theory and Its Applications, IEICE*, vol. 15, no. 2, pp. 273–283, 2024.

- [9] G. Carleo and M. Troyer, “Solving the quantum many-body problem with artificial neural networks,” *Science*, vol. 355, no. 6325, pp. 602–606, 2017.
- [10] S. Lu, X. Gao, and L.-M. Duan, “Efficient representation of topologically ordered states with restricted boltzmann machines,” *Physical Review B*, vol. 99, no. 15, p. 155136, 2019.
- [11] K. Hatakeyama-Sato, H. Adachi, M. Umeki, T. Kashikawa, K. Kimura, and K. Oyaizu, “Automated design of li+-conducting polymer by quantum-inspired annealing,” *Macromolecular Rapid Communications*, vol. 43, no. 20, p. 2200385, 2022.
- [12] G. E. Hinton, “Training products of experts by minimizing contrastive divergence,” *Neural Computation*, vol. 14, no. 8, pp. 1771–1800, 2002.
- [13] T. Tieleman, “Training restricted boltzmann machines using approximations to the likelihood gradient,” in *Proceedings of the 25th international conference on Machine learning*, pp. 1064–1071, 2008.
- [14] R. H. Swendsen and J.-S. Wang, “Replica monte carlo simulation of spin-glasses,” *Physical Review Letters*, vol. 57, no. 21, p. 2607, 1986.
- [15] K. Hukushima and K. Nemoto, “Exchange monte carlo method and application to spin glass simulations,” *Journal of the Physical Society of Japan*, vol. 65, no. 6, pp. 1604–1608, 1996.
- [16] E. Marinari and G. Parisi, “Simulated tempering: A new monte carlo scheme,” *Europhysics Letters*, vol. 19, p. 451, jul 1992.
- [17] R. M. Neal, “Sampling from multimodal distributions using tempered transitions,” *Statistics and Computing*, vol. 6, no. 4, p. 353–366, 1996.
- [18] G. Behrens, N. Friel, and M. Hurn, “Tuning tempered transitions,” *Statistics and Computing*, vol. 22, no. 1, p. 65–78, 2010.
- [19] G. Desjardins, A. Courville, Y. Bengio, P. Vincent, and O. Delalleau, “Tempered markov chain monte carlo for training of restricted boltzmann machines,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* (Y. W. Teh and M. Titterton, eds.), vol. 9 of *Proceedings of Machine Learning Research*, (Chia Laguna Resort, Sardinia, Italy), pp. 145–152, PMLR, 13–15 May 2010.
- [20] K. Cho, T. Raiko, and A. Ilin, “Parallel tempering is efficient for learning restricted boltzmann machines,” in *The 2010 International Joint Conference on Neural Networks (IJCNN)*, p. 1–8, IEEE, 2010.
- [21] G. Desjardins, H. Luo, A. Courville, and Y. Bengio, “Deep tempering,” 2014.
- [22] P. Smolensky, *Information Processing in Dynamical Systems: Foundations of Harmony Theory*, p. 194–281. Cambridge, MA, USA: MIT Press, 1986.
- [23] N. Le Roux and Y. Bengio, “Representational power of restricted boltzmann machines and deep belief networks,” *Neural Computation*, vol. 20, no. 6, pp. 1631–1649, 2008.
- [24] D. A. Levin, Y. Peres, and E. L. Wilmer, *Markov Chains and Mixing Times*. American Mathematical Society, 2 ed., 2017.
- [25] M. Yasuda, “Monte carlo integration using spatial structure of markov random field,” *Journal of the Physical Society of Japan*, vol. 84, no. 3, p. 034001, 2015.

- [26] M. Yasuda and K. Uchizawa, “A generalization of spatial monte carlo integration,” *Neural Computation*, vol. 33, pp. 1037–1062, 03 2021.
- [27] K. Sekimoto and M. Yasuda, “Effective learning algorithm for restricted boltzmann machines via spatial monte carlo integration,” *Nonlinear Theory and Its Applications, IEICE*, vol. 14, no. 2, pp. 228–241, 2023.
- [28] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, vol. 9 of *Proceedings of Machine Learning Research*, (Chia Laguna Resort, Sardinia, Italy), pp. 249–256, PMLR, 2010.
- [29] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017.
- [30] M. Yasuda and C. Takahashi, “Free energy evaluation using marginalized annealed importance sampling,” *Phys. Rev. E*, vol. 106, p. 024127, Aug 2022.
- [31] P. E. Jacob, J. O’Leary, and Y. F. Atchadé, “Unbiased markov chain monte carlo methods with couplings,” *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 82, pp. 543–600, 07 2020.

A Semi-Second-Order SMCI for RBMs

This appendix presents the specific formulations of semi-second-order SMCI for the expectations $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i]$, $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[h_j]$, and $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i h_j]$. These expectations are required to evaluate the parameter gradients in Eqs. (12)–(14). The estimators for $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i]$ and $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[h_j]$ are formulated as

$$\begin{aligned}\mathcal{M}_i^{(v)}(\mathbf{v}) &:= \mathbb{E}_{P_\theta(v_i, \mathbf{h} | \mathbf{v}_{-i})}[v_i] = \text{sig}[\phi_i(\mathbf{v}; \theta)], \\ \mathcal{M}_j^{(h)}(\mathbf{h}) &:= \mathbb{E}_{P_\theta(\mathbf{v}, h_j | \mathbf{h}_{-j})}[h_j] = \text{sig}[\psi_j(\mathbf{h}; \theta)],\end{aligned}$$

respectively, where $\mathbf{v}_{-i} \in \{0, 1\}^{n-1}$ and $\mathbf{h} \in \{0, 1\}^{m-1}$ denote the visible variables excluding v_i and the hidden variables excluding h_j , respectively, and

$$\begin{aligned}\phi_i(\mathbf{v}; \theta) &:= b_i + \sum_{j \in H} [\text{sfp}(\xi_{j,i}(\mathbf{v}; \theta) + W_{i,j}) - \text{sfp}(\xi_{j,i}(\mathbf{v}; \theta))], \\ \psi_j(\mathbf{h}; \theta) &:= c_j + \sum_{i \in V} [\text{sfp}(\eta_{i,j}(\mathbf{h}; \theta) + W_{i,j}) - \text{sfp}(\eta_{i,j}(\mathbf{h}; \theta))].\end{aligned}$$

Here, $\eta_{i,j}(\mathbf{h}; \theta) := \eta_i(\mathbf{h}; \theta) - W_{i,j}h_j$ and $\xi_{j,i}(\mathbf{v}, \theta) := \xi_j(\mathbf{v}, \theta) - W_{i,j}v_i$. Moreover, we consider two estimators for $\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i h_j]$ given by

$$\begin{aligned}\mathcal{C}_{i,j}^{(v)}(\mathbf{v}) &:= \mathbb{E}_{P_\theta(v_i, \mathbf{h} | \mathbf{v}_{-i})}[v_i h_j] = \text{sig}(\text{logit}\{\text{sig}[\xi_{j,i}(\mathbf{v}; \theta)] \text{sig}[\phi_i(\mathbf{v}; \theta)]\} + W_{i,j}), \\ \mathcal{C}_{i,j}^{(h)}(\mathbf{h}) &:= \mathbb{E}_{P_\theta(\mathbf{v}, h_j | \mathbf{h}_{-j})}[v_i h_j] = \text{sig}(\text{logit}\{\text{sig}[\eta_{i,j}(\mathbf{h}; \theta)] \text{sig}[\psi_j(\mathbf{h}; \theta)]\} + W_{i,j}),\end{aligned}$$

where $\text{logit}(x) := \ln(x/(1-x))$ denotes the logit function (i.e., the inverse function of the sigmoid function), and

$$\begin{aligned}\phi_{i,j}(\mathbf{v}; \theta) &:= \phi_i(\mathbf{v}; \theta) - [\text{sfp}(\xi_{j,i}(\mathbf{v}; \theta) + W_{i,j}) - \text{sfp}(\xi_{j,i}(\mathbf{v}; \theta))], \\ \psi_{j,i}(\mathbf{h}; \theta) &:= \psi_j(\mathbf{h}; \theta) - [\text{sfp}(\eta_{i,j}(\mathbf{h}; \theta) + W_{i,j}) - \text{sfp}(\eta_{i,j}(\mathbf{h}; \theta))].\end{aligned}$$

Given the sample points $\{(\mathbf{v}^{(\nu)}, \mathbf{h}^{(\nu)}) \mid \nu = 1, 2, \dots, N_s\}$, we approximate the expectations as follows:

$$\begin{aligned}\mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i] &\approx \frac{1}{N_s} \sum_{\nu=1}^{N_s} \mathcal{M}_i^{(v)}(\mathbf{v}^{(\nu)}), \\ \mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[h_j] &\approx \frac{1}{N_s} \sum_{\nu=1}^{N_s} \mathcal{M}_j^{(h)}(\mathbf{h}^{(\nu)}), \\ \mathbb{E}_{P_\theta(\mathbf{v}, \mathbf{h})}[v_i h_j] &\approx \frac{1}{N_s} \sum_{\nu=1}^{N_s} \frac{\mathcal{C}_{i,j}^{(v)}(\mathbf{v}^{(\nu)}) + \mathcal{C}_{i,j}^{(h)}(\mathbf{h}^{(\nu)})}{2},\end{aligned}$$

respectively.

B Transition Probability Matrix for BGS

In this appendix, we discuss the transition probability matrix for BGS. The relaxation time can be evaluated by estimating the second-largest eigenvalue of the transition probability matrix. We consider the transition probability matrix corresponding to the BGS kernel in Eq. (5), $\mathbf{P} \in \mathbb{R}^{2^n \times 2^n}$, whose elements are defined by $P_{a,b} := T_\theta^{\text{BGS}}(\mathbf{v}^{(b)} \mid \mathbf{v}^{(a)})$ for $a, b \in \{1, 2, \dots, 2^n\}$. Here, $\mathcal{V} := \{\mathbf{v}^{(1)}, \mathbf{v}^{(2)}, \dots, \mathbf{v}^{(2^n)}\}$ denotes an enumeration of all visible states in $\{0, 1\}^n$, and $\mathbf{v}^{(a)} \in \mathcal{V}$ is the a -th visible state. We define a matrix similar to $\mathbf{P}$ by $\mathbf{S} := \mathbf{D}^{1/2} \mathbf{P} \mathbf{D}^{-1/2}$, where

$$\mathbf{D} := \text{diag}(P_\theta(\mathbf{v}^{(1)}), P_\theta(\mathbf{v}^{(2)}), \dots, P_\theta(\mathbf{v}^{(2^n)})).$$

Here, $\mathbf{D}^{1/2}$ and $\mathbf{D}^{-1/2}$ are diagonal matrices whose a -th diagonal entries are $\sqrt{P_\theta(\mathbf{v}^{(a)})}$ and $1/\sqrt{P_\theta(\mathbf{v}^{(a)})}$, respectively. Since the probability matrix $\mathbf{P}$ satisfies the detailed balance condition (i.e., $\mathbf{D}\mathbf{P} = \mathbf{P}^\top \mathbf{D}$), we have

$$\mathbf{S}^\top = \mathbf{D}^{-1/2} \mathbf{P}^\top \mathbf{D}^{1/2} = \mathbf{D}^{-1/2} \mathbf{D} \mathbf{P} \mathbf{D}^{-1/2} = \mathbf{S}.$$

Moreover, for any real vector $\mathbf{x} \in \mathbb{R}^{2^n}$, we have

$$\begin{aligned}\mathbf{x}^\top \mathbf{S} \mathbf{x} &= \sum_{a=1}^{2^n} \sum_{b=1}^{2^n} x_a x_b \sqrt{\frac{P_\theta(\mathbf{v}^{(a)})}{P_\theta(\mathbf{v}^{(b)})}} \sum_{\mathbf{h}} P_\theta(\mathbf{v}^{(b)} \mid \mathbf{h}) P_\theta(\mathbf{h} \mid \mathbf{v}^{(a)}) \\ &= \sum_{a=1}^{2^n} \sum_{b=1}^{2^n} x_a x_b \sqrt{\frac{P_\theta(\mathbf{v}^{(a)})}{P_\theta(\mathbf{v}^{(b)})}} \sum_{\mathbf{h}} P_\theta(\mathbf{v}^{(b)} \mid \mathbf{h}) \frac{P_\theta(\mathbf{v}^{(a)} \mid \mathbf{h}) P_\theta(\mathbf{h})}{P_\theta(\mathbf{v}^{(a)})} \\ &= \sum_{\mathbf{h}} P_\theta(\mathbf{h}) \left(\sum_{a=1}^{2^n} x_a \frac{P_\theta(\mathbf{v}^{(a)} \mid \mathbf{h})}{\sqrt{P_\theta(\mathbf{v}^{(a)})}} \right)^2 \geq 0.\end{aligned}$$

Therefore, $\mathbf{S}$ is symmetric and positive semidefinite, meaning that all eigenvalues of $\mathbf{S}$ are nonnegative real numbers. As any probability matrix has an eigenvalue equal to 1, the eigenvalues of $\mathbf{S}$ can be ordered in descending order as

$$1 = \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{2^n} \geq 0. \quad (26)$$

As $\mathbf{S}$ is similar to $\mathbf{P}$, the eigenvalues of $\mathbf{P}$ are also given by Eq. (26). The relaxation time of BGS is defined as

$$t_{\text{rel}} := \frac{1}{1 - \lambda_2}.$$

We next consider the computation of the second-largest eigenvalue λ_2 , which is used to evaluate the relaxation time. Here, for $\mathbf{u}_1 := \mathbf{D}^{1/2} \mathbf{1}_{2^n}$, we have $\mathbf{S}\mathbf{u}_1 = \mathbf{u}_1$ and $\mathbf{u}_1^\top \mathbf{u}_1 = 1$. Therefore, $\mathbf{u}_1$ is the eigenvector of $\mathbf{S}$ corresponding to the largest eigenvalue λ_1 . By Rayleigh–Ritz theorem, the eigenvector corresponding to the second-largest eigenvalue λ_2 , $\mathbf{u}_2$, is obtained as

$$\mathbf{u}_2 = \arg \max_{\substack{\mathbf{q} \in \mathbb{R}^{2^n} \\ \mathbf{q}^\top \mathbf{q} = 1 \\ \mathbf{q}^\top \mathbf{u}_1 = 0}} \mathbf{q}^\top \mathbf{S} \mathbf{q}.$$

The corresponding eigenvalue is $\lambda_2 = \mathbf{u}_2^\top \mathbf{S} \mathbf{u}_2$. We denote the orthogonal projection matrix that removes the component in the direction of $\mathbf{u}_1$ as $\mathbf{\Pi}_\perp := \mathbf{I}_{2^n} - \mathbf{u}_1 \mathbf{u}_1^\top$, and the orthogonality constraint for $\mathbf{u}_1$ can be then eliminated as

$$\mathbf{u}_2 = \arg \max_{\substack{\mathbf{q} \in \mathbb{R}^{2^n} \\ \mathbf{q}^\top \mathbf{q} = 1}} \mathbf{q}^\top \mathbf{\Pi}_\perp \mathbf{S} \mathbf{\Pi}_\perp \mathbf{q}. \quad (27)$$

Thus, $\mathbf{u}_2$ can be obtained as an eigenvector corresponding to the largest eigenvalue of $\mathbf{\Pi}_\perp \mathbf{S} \mathbf{\Pi}_\perp$.

Hereafter, we describe how λ_2 was estimated in the numerical experiment in Section 2.3. We first preprocess $\mathbf{D}$ and $\mathbf{\Pi}_\perp$. We subsequently evaluate the transition probability matrix $\mathbf{P}$ using a sampling approximation. For each visible state $\mathbf{v}^{(a)} \in \mathcal{V}$, we independently perform $N_2 = 2^{15}$ one-step BGS transitions:

$$\mathbf{h}_i^{(a)} \sim P_\theta(\mathbf{h} \mid \mathbf{v}^{(a)}), \quad \mathbf{v}_i^{(a)} \sim P_\theta(\mathbf{v} \mid \mathbf{h}_i^{(a)}),$$

for $i = 1, 2, \dots, N_2$. The transition probability from $\mathbf{v}^{(a)}$ to $\mathbf{v}^{(b)}$ is then estimated by

$$\hat{P}_{a,b} := \frac{1}{N_2} \sum_{i=1}^{N_2} \delta(\mathbf{v}_i^{(a)}, \mathbf{v}^{(b)}).$$

We then estimate the matrix $\mathbf{A} := \mathbf{D} \mathbf{P}$, which is symmetric because BGS satisfies the detailed balance condition, as

$$\hat{\mathbf{A}} := \frac{1}{2} \left(\mathbf{D} \hat{\mathbf{P}} + (\mathbf{D} \hat{\mathbf{P}})^\top \right).$$

Since $\mathbb{E}[\hat{\mathbf{P}}] = \mathbf{P}$ and $\mathbf{A} = \mathbf{A}^\top$, it holds $\mathbb{E}[\hat{\mathbf{A}}] = \mathbf{A}$. Thus, $\hat{\mathbf{A}}$ is a symmetric unbiased estimator of $\mathbf{A}$. Using $\hat{\mathbf{A}}$, the estimator of $\mathbf{S}$ is obtained by $\hat{\mathbf{S}} := \mathbf{D}^{-1/2} \hat{\mathbf{A}} \mathbf{D}^{-1/2}$, and we estimate $\mathbf{u}_2$ by obtaining the eigenvector corresponding to the largest eigenvalue of $\mathbf{\Pi}_\perp \hat{\mathbf{S}} \mathbf{\Pi}_\perp$. Using the resulting eigenvector $\hat{\mathbf{u}}_2$, we estimate λ_2 by $\hat{\lambda}_2 := \hat{\mathbf{u}}_2^\top \hat{\mathbf{S}} \hat{\mathbf{u}}_2$. Note that the resulting estimator $\hat{\lambda}_2$ is generally biased and is not guaranteed to provide either a lower or an upper bound on λ_2 because the estimator $\hat{\mathbf{S}}$ is used both to determine $\hat{\mathbf{u}}_2$ and to evaluate its Rayleigh quotient.

C Invariant Distribution of the Proposed Transition Kernel

We prove that the proposed transition kernel T_Θ^{prop} leaves $P_\Theta(\mathbf{V})$ invariant. We consider the product distribution from the ℓ -th RBM to the topmost RBM defined by $\Pi_\ell(\mathbf{V}_{\ell:L}) := \prod_{i=\ell}^L P_{\theta_i}(\mathbf{v}_i)$. First, the topmost kernel T_L leaves Π_L invariant because T_L is one step of BGS and satisfies the detailed balance

condition for Π_L . Next, we fix $\ell \in \{0, \dots, L-1\}$ and assume that $T_{\ell+1}$ satisfies the balance condition for $\Pi_{\ell+1}$, i.e.,

$$\sum_{\mathbf{V}_{\ell+1:L}} T_{\ell+1}(\mathbf{V}'_{\ell+1:L} \mid \mathbf{V}_{\ell+1:L}) \Pi_{\ell+1}(\mathbf{V}_{\ell+1:L}) = \Pi_{\ell+1}(\mathbf{V}'_{\ell+1:L}). \quad (28)$$

We then prove that the kernel T_ℓ leaves Π_ℓ invariant by following the sampling procedure of T_ℓ in Eq. (23) step by step. First, sampling from $P_{\theta_\ell}(\mathbf{h}_\ell \mid \mathbf{v}_\ell)$ gives

$$\sum_{\mathbf{v}_\ell} P_{\theta_\ell}(\mathbf{h}_\ell \mid \mathbf{v}_\ell) \Pi_\ell(\mathbf{V}_{\ell:L}) = P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell) \Pi_{\ell+1}(\mathbf{V}_{\ell+1:L}).$$

Since the swap kernel $S_{\ell,\ell+1}$ leaves $P_{\theta_\ell}^{(h)} P_{\theta_{\ell+1}}^{(v)}$ invariant, applying the swap kernel gives

$$\begin{aligned} \sum_{\mathbf{h}_\ell} \sum_{\mathbf{v}_{\ell+1}} S_{\ell,\ell+1}(\mathbf{h}_\ell^+, \mathbf{v}_{\ell+1}^+ \mid \mathbf{h}_\ell, \mathbf{v}_{\ell+1}) P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell) P_{\theta_{\ell+1}}^{(v)}(\mathbf{v}_{\ell+1}) \Pi_{\ell+2}(\mathbf{V}_{\ell+2:L}) \\ = P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell^+) \Pi_{\ell+1}(\mathbf{v}_{\ell+1}^+, \mathbf{V}_{\ell+2:L}). \end{aligned}$$

Applying $T_{\ell+1}$ and using Eq. (28) yields

$$\begin{aligned} \sum_{\mathbf{v}_{\ell+1}^+} \sum_{\mathbf{V}_{\ell+2:L}} T_{\ell+1}(\hat{\mathbf{v}}_{\ell+1}, \mathbf{V}'_{\ell+2:L} \mid \mathbf{v}_{\ell+1}^+, \mathbf{V}_{\ell+2:L}) P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell^+) \Pi_{\ell+1}(\mathbf{v}_{\ell+1}^+, \mathbf{V}_{\ell+2:L}) \\ = P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell^+) \Pi_{\ell+1}(\hat{\mathbf{v}}_{\ell+1}, \mathbf{V}'_{\ell+2:L}). \end{aligned}$$

Applying the swap kernel in the downward direction gives

$$\begin{aligned} \sum_{\mathbf{h}_\ell^+} \sum_{\hat{\mathbf{v}}_{\ell+1}} S_{\ell,\ell+1}(\mathbf{h}_\ell^-, \mathbf{v}'_{\ell+1} \mid \mathbf{h}_\ell^+, \hat{\mathbf{v}}_{\ell+1}) P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell^+) \Pi_{\ell+1}(\hat{\mathbf{v}}_{\ell+1}, \mathbf{V}'_{\ell+2:L}) \\ = P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell^-) \Pi_{\ell+1}(\mathbf{V}'_{\ell+1:L}). \end{aligned}$$

Finally, sampling from $P_{\theta_\ell}(\mathbf{v}_\ell \mid \mathbf{h}_\ell^-)$ gives

$$\sum_{\mathbf{h}_\ell^-} P_{\theta_\ell}(\mathbf{v}'_\ell \mid \mathbf{h}_\ell^-) P_{\theta_\ell}^{(h)}(\mathbf{h}_\ell^-) \Pi_{\ell+1}(\mathbf{V}'_{\ell+1:L}) = \Pi_\ell(\mathbf{V}'_{\ell:L}).$$

Therefore, T_ℓ leaves Π_ℓ invariant. By mathematical induction, the proposed transition kernel leaves $\Pi_0(\mathbf{V})$ (i.e., $P_\Theta(\mathbf{V})$) invariant, as does the DT kernel.